

UNIVERSIDAD NACIONAL DE EL SALVADOR

FACULTAD DE INGENIERÍA Y ARQUITECTURA

Guía Práctica N° 1: Programación Orientada a Objetos (POO) en C#

MATERIA: Desarrollo y Reutilización de Software / IDS

PRESENTADO POR:

- José Cerritos EE22004

- Wilian Salinas SV99004

DOCENTE: Ing. Dinora Osorio.

GRUPO: 2

REPOSITORIO OFICIAL DEL PROYECTO (GITHUB):

<https://github.com/Jh00z3p27/Solucion-Guia-1-DRS-JW>

FECHA DE ENTREGA: 06 de Septiembre de 2026.

RESUMEN EXPLICATIVO

El presente trabajo aborda la implementación práctica de los pilares fundamentales de la Programación Orientada a Objetos (POO) utilizando el lenguaje de programación C# bajo el entorno de ejecución .NET SDK.

A continuación se describe la solución desarrollada para cada uno de los ejercicios que componen la guía de laboratorio:

1. EJERCICIO 1: ABSTRACCIÓN DE DATOS (CuentaBancariaJW)

Concepto Aplicado:

La Abstracción consiste en aislar los detalles complejos de la lógica interna de un sistema y exponer únicamente la interfaz pública necesaria para operar sobre él.

Implementación:

Se diseñó la clase 'CuentaBancariaJW' en la cual la variable del saldo ('saldoJW') permanece oculta bajo el modificador de acceso 'private'. La modificación del saldo no se realiza directamente, sino a través de métodos públicos ('DepositarJW' y 'RetirarJW') que aplican las reglas de negocio, como validar montos positivos y comprobar la disponibilidad de fondos antes de efectuar una transacción.

2. EJERCICIO 2: ENCAPSULACIÓN (EmpleadoJW)

Concepto Aplicado:

La Encapsulación protege el estado interno de un objeto impidiendo la alteración no autorizada o inconsistente de sus atributos desde el exterior.

Implementación:

Se creó la clase 'EmpleadoJW' haciendo uso de propiedades de C# (bloques 'get' y 'set'). El atributo 'salarioJW' se encapsula mediante la propiedad pública 'SalarioJW', la cual incluye lógica de

validación dentro del bloque 'set' para asegurar que el salario asignado a un empleado sea un valor no negativo, bloqueando automáticamente cualquier intento de modificación inválida.

3. EJERCICIO 3: HERENCIA SIMPLE (VehiculoJW -> CocheJW)

Concepto Aplicado:

La Herencia permite crear clases derivadas a partir de una clase base, compartiendo atributos y métodos generales para promover la reutilización de código.

Implementación:

Se definió la clase base 'VehiculoJW' con atributos generales ('MarcaJW', 'ModeloJW') y el método 'ArrancarJW()'. Posteriormente, la clase 'CocheJW' heredó de la clase base mediante el operador de herencia ':', añadiendo sus propios atributos específicos ('NumPuertasJW') y métodos especializados ('AbrirMaleteroJW()'), reutilizando el constructor padre a través de la palabra clave 'base'.

4. EJERCICIO 4: POLIMORFISMO (AnimalJW -> PerroJW / GatoJW)

Concepto Aplicado:

El Polimorfismo es la capacidad que tienen diferentes objetos de responder a una misma llamada o método de formas distintas según su implementación concreta.

Implementación:

Se diseñó la clase base 'AnimalJW' con un método virtual 'HacerSonidoJW()'. Las clases derivadas 'PerroJW' y 'GatoJW' redefinieron el comportamiento de dicho método utilizando la palabra clave 'override'. Mediante una lista genérica de tipo 'AnimalJW', se demostró cómo el sistema procesa diferentes animales de forma uniforme ejecutando la respuesta específica de cada uno.

5. EJERCICIO 5: HERENCIA MULTINIVEL (AnimalBaseJW -> MamiferoJW -> PerroMultinivelJW)

Concepto Aplicado:

La Herencia Multinivel establece una jerarquía en cadena donde una clase derivada actúa como clase base para una nueva subclase.

Implementación:

Se creó una estructura de tres niveles:

- Nivel 1 (Clase Raíz): 'AnimalBaseJW' con el método 'ComerJW()'.
- Nivel 2 (Clase Intermedia): 'MamiferoJW' que hereda del Nivel 1 y añade 'AmamantarJW()'.
- Nivel 3 (Clase Hoja): 'PerroMultinivelJW' que hereda del Nivel 2 y añade 'LadRARJW()'.

Se comprobó que el objeto final del Nivel 3 posee acceso acumulativo a todas las funcionalidades de los niveles superiores de la jerarquía.